

Bill Joy and is free software released under the BSD license. Notice that the C shell is also not pre-installed in Ubuntu. But just like the installation of KornShell and the Z shell, installing the C shell also required just one command: 'sudo apt install csh'. This installation too was very smooth. The C shell is usually invoked with the command *csh*. This is a shell that is somewhat different from the four discussed earlier.

From Figure 5, we can see that the command 'echo \$0' to print the name of the current shell doesn't work with the C shell. So the command 'ps -p \$\$' is used to print the name of the current shell. Notice that this is the first time a shell has displayed behaviour that is different from the others. Figure 5 also shows some basic commands like 'pwd' and 'date' executed on the C shell terminal. From Figure 5 we can see that on executing the command 'exit', the default shell Bash is ready again for operation. The C shell has an advanced version called tcsh, which can be invoked with the command 'tcsh'. But you need to install tcsh also with the command 'sudo apt install tcsh'. Figure 5 also shows the tcsh shell invoked and the command 'pwd' executed on it. From the figure, we can see that the commands 'echo \$0' and 'exit' also work as expected.

fish

Now let us discuss one more shell

```
File Edit View Search Terminal Help
# csh
% echo $0
No file for $0.
% ps -p $$
  PID TTY          TIME CMD
 21391 pts/0    00:00:00 csh
% pwd
/home
% date
Sun Jun  7 23:43:51 IST 2020
% exit
% exit
# echo $0
bash
# tcsh
deepu:/home> echo $0
tcsh
deepu:/home> pwd
/home
deepu:/home> exit
exit
# echo $0
bash
```

Figure 5: Using the C shell and tcsh shell in Linux

called *fish* that can be used in Linux. Of course, I could have kept on extending this list by installing and testing obscure shells with no distinctive features. All the other shells were chosen for historic reasons or current impact. I might have missed a few important ones – I don't know. But *fish* was included in this list for a third reason. While researching for this article, I accidentally stumbled upon the official website of the *fish* shell. The site claims that the shell is ready for the 90s! The tone made me think that the shell had been developed in the late 80s and the team was being futuristic! Actually, that was not the case. The team behind *fish* was being very humble. Though it was first released in the mid-2000s, *fish* claimed to be behind the times and only ready for the 90s! This fun way of advertising the features of this shell caught my attention. *fish* is free and open source software released under the GNU General Public License.

With claimed features like 'Auto suggestions', 'Glorious VGA Color', 'Web based configuration', etc, the *fish* shell looks promising. So, I felt like including *fish* in this list of Linux shells. Notice that the *fish* shell is also not pre-installed in Ubuntu. But it too can be installed very easily. Figure 6 shows one important feature of the *fish* shell, 'Auto suggestions'. It can be seen from the figure that by just typing the letter 'a', the command 'apt-get update' gets offered as a suggestion.

Now let us compare these six shells a bit more. Since a shell is just an interface that allows the user to communicate with the underlying kernel of the operating system, while using basic Linux commands we can't expect major differences because of the execution of the underlying Linux system calls. But one major area

```
File Edit View Search Terminal Help
# fish
Welcome to fish, the friendly interactive
deepu@deepu ~-> aapt-get update
```

Figure 6: Using the *fish* shell in Linux

where the various shells could behave differently is in their shell script syntax. Let us consider the simple Bourne shell script called *bshell.sh* shown below, which checks whether the number entered through the keyboard is odd or even. Of course, this shell script called *bshell.sh* is very simple, but it is sufficient to test the syntax of the various shells we have discussed in this article.

```
echo "Enter a Number "
read num
rem=`expr $num % 2`
if [ $rem -eq 0 ]
then
    echo "Even Number"
else
    echo "Odd Number"
fi
```

Figure 7 shows the output of the shell script *bshell.sh* when executed by the following different shells — the Bourne shell, Bash, KornShell, mksh and the Z shell. From Figure 7, it is clear that they were all able to execute the shell script correctly without any errors. So we can conclude that the Bourne shell, Bash, KornShell, mksh and the Z shell have similar syntax. Of course, there could be minute differences between all of them. Figure 8 shows the execution of the shell script *bshell.sh* by the C shell, tcsh, and *fish*. From Figure 8, it is clear that all three of them failed to execute the script *bshell.sh*.

```
File Edit View Search Terminal Help
# sh bshell.sh
Enter a Number
111
Odd Number
# bash bshell.sh
Enter a Number
222
Even Number
# ksh bshell.sh
Enter a Number
333
Odd Number
# mksh bshell.sh
Enter a Number
444
Even Number
# zsh bshell.sh
Enter a Number
555
```

Figure 7: Successful execution of the script *bshell.sh*